NexoEventos

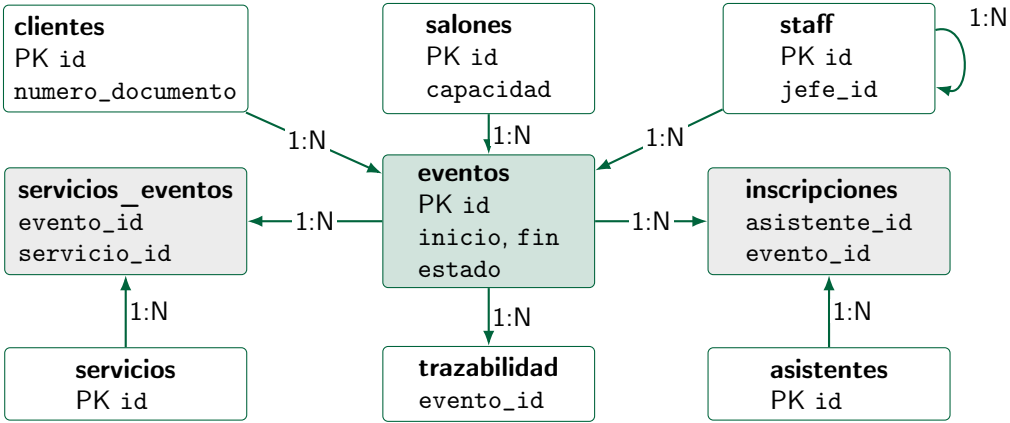
Frontend, reglas de negocio y demostración

Antonio Gutiérrez, David Rodríguez,
David Pascagaza y Sara Torres

Ingeniería de Datos, Universidad del Rosario

Octubre de 2026

Esquema de la base de datos



Restricciones por entidad

clientes

- **U** numero_documento
- **C** numero_documento: solo dígitos, máx. 10
- **C** tipo_documento ∈ {CC, NIT}
- **C** email: formato válido
- **C** telefono: solo dígitos, máx. 10

salones

- **C** tamano ∈ {Pequeño, Mediano, Grande}
- **C** capacidad > 0
- **C** precio_hora >= 0

eventos

- **C** inicio < fin
- **C** aforo_esperado > 0
- **C** estado ∈ {Cotizado, Confirmado, Finalizado, Cancelado}
- **C** tipo ∈ {Congreso, Conferencia, Taller, Feria, Seminario, Exposición, Corporativo, Social, Otro}
- **C** precio_hora_salon_cotizado >= 0
- **C** valor_total_cotizado >= 0
- **C** tasa_asistencia entre 0 y 100
- **F** coordinador_id → staff
- **F** salon_id → salones
- **F** cliente_id → clientes

servicios

- **C** precio >= 0
- **C** unidad_cobro ∈ {Hora, Unidad, Persona}

trazabilidad

- **C** campo_modificado ∈ {estado, salon_id, inicio, fin}
- **F** evento_id → eventos

Leyenda

C CHECK **U** UNIQUE **F** FK

Restricciones por entidad

staff

- **C** `email`: formato válido
- **C** `area` $\in$ {Dirección, Operaciones, Comercial, Logística, Alimentos y Bebidas, Audiovisuales}
- **C** `cargo` $\in$ {Director, Gerente, Jefe, Coordinador, Auxiliar, Técnico}
- **C** `jefe_id IS NULL OR jefe_id <> id`
- **C** solo el Director tiene `jefe_id` nulo
- **F** `jefe_id` $\rightarrow$ `staff`

asistentes

- **U** `documento_identidad`
- **C** `documento_identidad`: solo dígitos, máx. 10
- **C** `email`: formato válido

inscripciones

- **U** (`asistente_id`, `evento_id`)
- **C** `fecha_checkin <= NOW()`
- **F** `asistente_id` $\rightarrow$ `asistentes`
- **F** `evento_id` $\rightarrow$ `eventos`

servicios_eventos

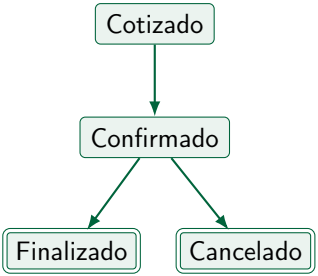
- **U** (`evento_id`, `servicio_id`)
- **C** `cantidad > 0`
- **C** `precio_unidad_cotizado >= 0`
- **F** `evento_id` $\rightarrow$ `eventos`
- **F** `servicio_id` $\rightarrow$ `servicios`

Leyenda

C CHECK **U** UNIQUE **F** FK

Diseño de los triggers: reglas en la base de datos

Ciclo de vida del evento



Estados finales inmodificables

A1 `tr_eventos_2_disponibilidad`
salón habilitado · aforo $\leq$ capacidad · cruce + 1 h de montaje · aforo $\geq$ inscritos

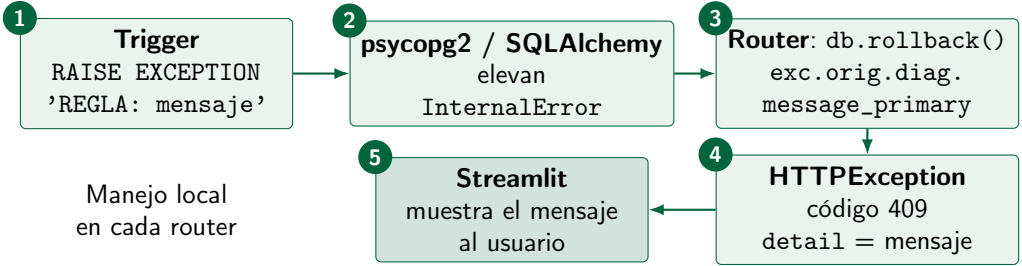
A2 `tr_inscripciones_cupo_checkin`
no Cancelado/Finalizado · cupo $\leq$ aforo · 1 check-in · solo Confirmado y en ventana horaria

A3 `tr_servicios_1_contratar`, `tr_servicios_2_total`, `tr_eventos_3_liquidacion`, `tr_eventos_5_servicios`
total = salón + servicios · precio de catálogo congelado

A4 `tr_eventos_1_estado`, `tr_eventos_4_auditoria`
estados finales fijos · tasa de asistencia · trazabilidad

A5 `fn_salones_disponibles(inicio, fin, personas)`
salones libres en un rango y aforo

Cómo la API traduce los errores de los triggers



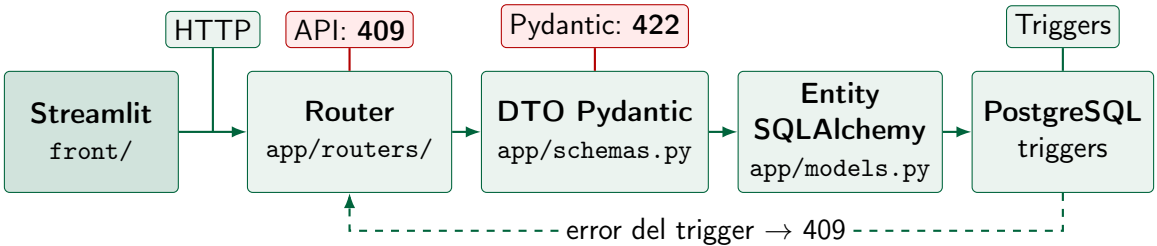
Códigos de respuesta

200/201/204	éxito
404	recurso no existe
409	**regla violada (trigger)**
422	DTO inválido (Pydantic)

Respuesta ilustrativa (formato esperado)

```
{"detail": "DISPONIBILIDAD: El salón
\"X\" se cruza con el evento ... Se
requiere 1 hora libre entre eventos."}
```

Del clic en Streamlit a la base de datos



El frontend **nunca** se conecta directo a la base: toda operación pasa por la API (10 routers, un DTO por recurso).

Frontend en Streamlit: 5 módulos + tablero

Módulos (menú lateral)

- **Eventos**: crear, cotización, servicios, estado, reprogramar, auditoría
- **Asistentes y check-in**: registro, inscripción, llegada
- **Disponibilidad de salones**: por horario y aforo
- **Clientes y staff**: registro y consulta
- **Reportes**: B1–B4 con tablas y gráficos

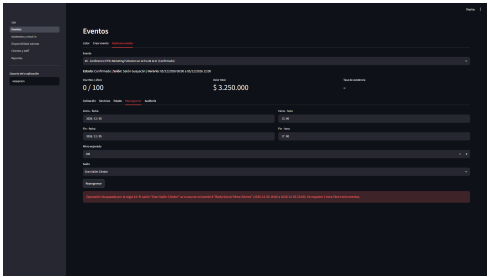


Video 1: recorrido funcional

1 Regla bloqueada: cruce de horario

Eventos > Gestionar > Reprogramar:
mismo salón y horario encimado con otro evento

→ `Operación bloqueada por la regla A1: DISPONIBILIDAD...`



Tablero · disponibilidad de salones (A5) · crear evento · cotización y servicios (A3) · estado y auditoría (A4)

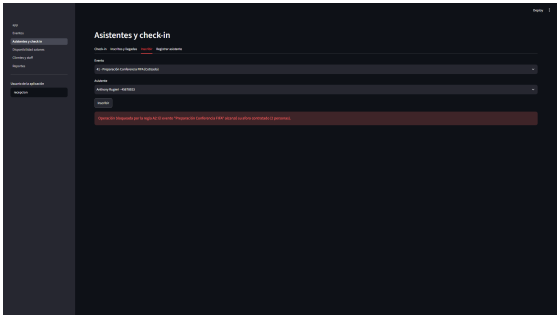
▸ Prueba 1: Streamlit (Video)

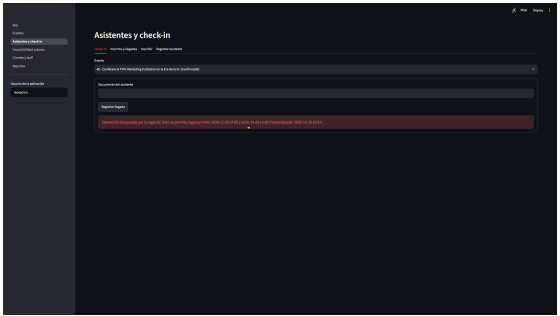
Video 2: reglas bloqueando operaciones

2 Regla bloqueada: cupo lleno

Evento demo con aforo 2, luego **Asistentes y check-in** > Inscribir: 1.° y 2.° entran, el 3.°...

→ `Operación bloqueada por la regla A2: CUPO...`





A1: cruce de horario al reprogramar · A2: cupo lleno al inscribir

▸ Prueba 2: Streamlit (Video)

Video 3: reportes del Módulo B

3 Reportes (Módulo B)

- **B1 Organigrama**
  CTE recursiva
- **B2 Escalamiento**
  CTE recursiva
- **B3 Pareto de clientes**
  funciones de ventana
- **B4 Ocupación de salones**
  `LAG()`

B1 organigrama · B2 escalamiento · B3 Pareto de clientes · B4 ocupación de salones

▸ Prueba 3: Streamlit (Video)